

Structures de Données Avancées

Pseudocode & Analyse de Complexité

Pr. Khadija SLIMANI

khadija.slimani@esiea.fr

PASS Java — Année universitaire 2026–2027

Plan du cours

Séance 7 — Structures Linéaires

1. Listes à taille variable : Allocation dynamique et réallocation.
2. Listes chaînées : Nœuds, pointeurs, chaînage simple et double.
3. Opérations : Insertion, suppression, parcours séquentiel.
4. Quand utiliser des listes vs tableaux ?

Séance 8 — Structures Hiérarchiques

5. Arbres binaires : Vocabulaire, propriétés et structures.
6. Parcours : Préfixe, Infixe, Postfixe en récursif.
7. ABR / BST : Propriété d'ordre et recherche dichotomique.
8. Intérêt : Optimisation des accès en $O(\log n)$.

Pourquoi étudier les structures de données ?



Gestion Mémoire

Contrôler la quantité de mémoire allouée et éviter le gaspillage lors de la manipulation de grands volumes.



Vitesse d'Exécution

Réduire la complexité temporelle des algorithmes de recherche, d'insertion et de suppression.



Modélisation Réelle

Adapter la structure de données à la nature du problème (file d'attente, arborescence, réseaux).

Séance 7

Structures Linéaires Avancées

Listes dynamiques, Listes chaînées, Pointeurs et analyse des cas d'usage.

Tableaux statiques : une taille fixée à la compilation

Caractéristiques du tableau statique

- Taille réservée à la déclaration : T[100].
- Impossible d'agrandir si le nombre d'éléments dépasse 100.
- Mémoire gaspillée si l'on n'utilise que 5 éléments.

Le problème pratique

- Dans la majorité des applications réelles, la quantité exacte de données à traiter est inconnue à l'avance.

```
// Déclaration Statique
Variables
  T : Tableau [1..100] de Entier

// Si n = 150 : Erreur Débordement !
// Si n = 3 : 97 cases inutilisées.
```

Concept de la liste dynamique (Tableau redimensionnable)

Une **liste dynamique** est un tableau interne qui s'agrandit ou se réduit automatiquement selon le besoin de l'application.

Deux Notions Distinctes

1. **Taille (Size)** : Nombre réel d'éléments actuellement stockés.
2. **Capacité (Capacity)** : Taille totale du bloc mémoire alloué en RAM.

```
// Structure d'une Liste Dynamique
Structure    ListeDynamique
    donnees : Tableau de    Entier
    taille  : Entier
    capacite : Entier
FinStructure
```

Mécanisme de redimensionnement (Doublement de capacité)

Étapes de la Réallocation

1. Vérifier si `taille == capacite`.
2. Allouer un nouveau bloc mémoire de taille `capacite * 2`.
3. Copier tous les éléments de l'ancien tableau vers le nouveau.
4. Libérer l'ancien bloc mémoire.

```
Procédure redimensionner (L : ListeDynamique )  
    nouvelleCapacite ← L.capacite * 2  
    nouveauBloc ← allouerTableau (nouvelleCapacite)  
    Pour i de 0 à L.taille - 1 Faire  
        nouveauBloc[i] ← L.donnees[i]  
    FinPour  
    liberer (L.donnees)  
    L.donnees ← nouveauBloc  
    L.capacite ← nouvelleCapacite  
FinProcédure
```

Accès direct aux éléments : La force des tableaux

Calcul d'adresse en $O(1)$

En raison de la **contiguïté mémoire**, l'adresse de $L[i]$ est calculée instantanément :

$$\text{Adresse}(L[i]) = \text{AdresseBase} + (i \times \text{TailleType})$$

Conséquence sur la performance

Que la liste contienne 10 éléments ou 10 millions d'éléments, lire $L[500]$ prend exactement le même temps.

Complexité de l'accès direct : $O(1)$.

Insertion en fin de liste dynamique

```
Procédure ajouterFin (L : ListeDynamique , val : Entier )  
  Si L.taille = L.capacite Alors  
    redimensionner (L)  
  FinSi  
  
  L.donnees[L.taille] ← val  
  L.taille ← L.taille + 1  
FinProcédure
```

Analyse : Complexité Amortie

- Cas normal (espace dispo) : **$O(1)$** .
- Cas rare (tableau plein) : Copie de n éléments en **$O(n)$** .

En moyenne sur une série d'ajouts, la complexité moyenne est appelée **$O(1)$ amorti**.

Insertion à un index quelconque : Le coût du décalage

```
Procédure insérerA (L : ListeDynamique , val : Entier , idx :  
Entier )  
  Si L.taille = L.capacite Alors  
    redimensionner (L)  
  FinSi  
  
  // Décalage des éléments vers la droite  
  Pour i de L.taille à idx + 1 par pas de -1 Faire  
    L.donnees[i] ← L.donnees[i - 1]  
  FinPour  
  
  L.donnees[idx] ← val  
  L.taille ← L.taille + 1  
FinProcédure
```

Pire Cas : Insertion au Début (idx = 0)

Insérer en première position impose de décaler **tous les n éléments** d'une case vers la droite.

Complexité temporelle : $O(n)$.

Suppression à un index et tassement

```
Procédure supprimerA (L : ListeDynamique , idx : Entier )  
  // Décalage des éléments vers la gauche  
  Pour i de idx à L.taille - 2 Faire  
    L.donnees[i] ← L.donnees[i + 1]  
  FinPour  
  
  L.taille ← L.taille - 1  
FinProcédure
```

Maintien du Trou

Pour ne pas laisser de "case vide" au milieu du tableau, il faut re-tasser tous les éléments suivants.

- Suppression à la fin : **$O(1)$** .
- Suppression au début : **$O(n)$** .

Limites principales des structures contiguës



Fragmentation mémoire

Nécessite un grand bloc d'octets ininterrompu.



Lenteur des insertions

Décalages coûteux en $O(n)$ pour les insertions en tête.



Surcoût de copie

Copie intégrale du tableau lors de chaque réallocation.

Les Listes Chaînées : Allocation Disperse

Une **Liste Chaînée** stocke ses éléments dans des **nœuds indépendants** dispersés en mémoire, reliés par des **pointeurs**.

Plus de contiguïté requise

Chaque nœud est alloué là où il y a de la place disponible en RAM.



Anatomie d'un Nœud (Node)

Champ 1 : La Donnée (Data)

Contient la valeur réelle à stocker (Entier, Chaîne, Objet complexe).

Champ 2 : Le Pointeur (Next)

Contient l'adresse mémoire du nœud suivant dans la liste (ou NULL si dernier).

Définition des Structures de Données en Pseudocode

```
// Définition d'un Nœud
Structure  Nœud
  valeur :  Entier
  suivant : Pointeur sur Nœud
FinStructure

// Structure de la Liste
Structure  ListeChaine
  tete :  Pointeur sur Nœud
  taille : Entier
FinStructure
```

Le Rôle de la tête (Head)

La variable tete est le seul point d'entrée de la liste.

- Si tete == NULL, la liste est considérée comme vide.

Parcours Séquentiel d'une Liste Chaînée

```
Procédure afficherListe (L : ListeChaine )  
    courant : Pointeur sur Noeud ← L.tete  
  
    TantQue courant ≠ NULL Faire  
        afficher (courant.valeur)  
        courant ← courant.suivant // Avancer  
    FinTantQue  
FinProcédure
```

Pas d'accès direct

Pour accéder au 100ème élément, il faut obligatoirement traverser les 99 premiers nœuds un par un.

Complexité de l'accès : $O(n)$.

Insertion en tête de liste : Un gain majeur en $O(1)$

```
Procédure insérerTete (L : ListeChaine , val : Entier )  
    nouveau : Pointeur sur Noeud ← nouveau Noeud()  
    nouveau.valeur ← val  
  
    // Raccordement rapide  
    nouveau.suivant ← L.tete  
    L.tete ← nouveau  
    L.taille ← L.taille + 1  
FinProcédure
```

Pourquoi est-ce $O(1)$?

Aucun élément n'est déplacé en mémoire. On modifie simplement 2 adresses pointeurs !

C'est l'un des avantages majeurs des listes chaînées par rapport aux tableaux.

Insertion en fin de liste chaînée

```
Procédure insérerFin (L : ListeChaine , val : Entier )
nouveau : Pointeur sur Noeud ← nouveau Noeud ()
nouveau.valeur ← val
nouveau.suivant ← NULL

Si L.tete = NULL Alors
    L.tete ← nouveau
Sinon
    courant : Pointeur sur Noeud ← L.tete
    TantQue courant.suivant ≠ NULL Faire
        courant ← courant.suivant
    FinTantQue
    courant.suivant ← nouveau
FinSi
FinProcédure
```

Complexité : $O(n)$

Il faut d'abord traverser toute la liste jusqu'au dernier nœud.

Astuce : Maintenir un pointeur queue permet de passer cette opération en $O(1)$.

Insertion après un nœud connu

```
Procédure insérerAprès (pred : Pointeur sur Noeud , val : Entier )  
  Si pred = NULL Alors Retourner FinSi  
  
  nouveau : Pointeur sur Noeud ← nouveau Noeud()  
  nouveau.valeur ← val  
  
  // Réalignement des pointeurs  
  nouveau.suivant ← pred.suivant  
  pred.suivant ← nouveau  
FinProcédure
```

Ordre des Operations

Attention : Il faut connecter nouveau.suivant AVANT de casser le lien pred.suivant, sous peine de perdre la fin de la liste !

Suppression d'un nœud par valeur

```
Procédure supprimerValeur (L : ListeChaine , val : Entier )
    courant : Pointeur sur Noeud ← L.tete
    preced : Pointeur sur Noeud ← NULL

    TantQue courant ≠ NULL et courant.valeur ≠ val Faire
        preced ← courant
        courant ← courant.suivant
    FinTantQue

    Si courant = NULL Alors Retourner FinSi // Non trouvé

    Si preced = NULL Alors
        L.tete ← courant.suivant // Suppression Tête
    Sinon
        preced.suivant ← courant.suivant // Contournement
    FinSi
    liberer (courant)
FinProcédure
```

Libération Mémoire

L'appel à `liberer()` est obligatoire dans les langages sans Garbage Collector (comme C ou C++) pour éviter les fuites mémoire.

Les listes doublement chaînées

```
Structure NoeudDouble  
  valeur : Entier  
  suivant : Pointeur sur NoeudDouble  
  precedent : Pointeur sur NoeudDouble  
FinStructure
```

Avantages clés

- Navigation bidirectionnelle (avant/arrière).
- Suppression d'un nœud en $O(1)$ si on possède son pointeur.
- *Inconvénient* : Surcoût mémoire (1 pointeur de plus par nœud).

Quand utiliser liste dynamique vs liste chaînée ?

Liste dynamique si :

- Vous avez besoin d'accès direct par index : $L[i]$.
- Vous faites principalement des lectures et ajouts en fin.
- Vous recherchez une excellente efficacité du cache CPU.

Liste chaînée si :

- Vous faites de nombreuses insertions/suppressions en tête.
- La taille totale est imprévisible et très variable.
- Vous implémentez des Piles ou des Files.

Séance 8

Arbres & Arbres Binaires de Recherche

Structures Hiérarchiques, Parcours Récursifs, ABR et Recherches Dichotomiques en $O(\log n)$.

Pourquoi dépasser les structures linéaires ?

Le Piège de la Recherche Linéaire $O(n)$

Dans un tableau non trié ou une liste chaînée, chercher un élément nécessite de parcourir les n éléments un à un. Pour 1 million de données, cela demande jusqu'à 1 million d'opérations !

La Solution : La Hiérarchie

En organisant les données sous forme d'arbre, on peut diviser le problème par 2 à chaque étape.

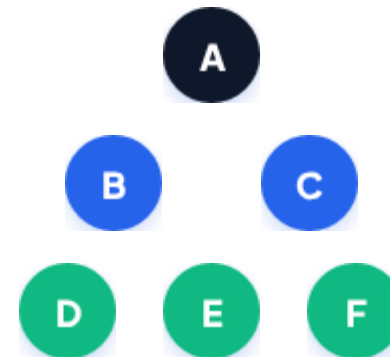
Pour 1 million de données, **20 comparaisons suffisent !**

Qu'est-ce qu'un arbre en algorithmique ?

Un **Arbre** est une structure non linéaire constituée d'un ensemble de **nœuds** reliés par des **arêtes** représentant des relations hiérarchiques.

Exemples Concrets

- Arborescence de fichiers du disque dur.
- DOM HTML d'une page Web.
- Index B-Tree des bases de données SQL.



Vocabulaire 1 : Racine, Feuilles et Nœuds internes

Sommet

La Racine

Nœud sommet unique sans aucun père (point d'entrée de l'arbre).

Extrémités

Les Feuilles

Nœuds n'ayant aucun fils (degré égal à 0).

Milieu

Nœuds Internes

Nœuds possédant à la fois un père et au moins un fils.

Vocabulaire 2 : Relations père, Fils et Frères

Père & Fils

- **Père** : Le nœud immédiatement supérieur.
- **Fils** : Les nœuds directement rattachés en dessous.
- Chaque nœud (sauf la racine) a **exactement un seul père**.

Frères & Ancêtres

- **Frères** : Nœuds partageant le même père.
- **Sous-arbre** : Arbre formé par un nœud et l'ensemble de ses descendants.

Vocabulaire 3 : Profondeur et Hauteur de l'arbre

Profondeur d'un Nœud

- Nombre d'arêtes reliant la racine à ce nœud.
- Racine : Profondeur 0.

Hauteur de l'Arbre (h)

- Profondeur maximale parmi toutes les feuilles de l'arbre.
- Détermine la performance maximale des algorithmes.

Qu'est-ce qu'un Arbre Binaire (Binary Tree) ?

Un **Arbre Binaire** est un arbre dans lequel chaque nœud a **au plus 2 fils** : le **fils gauche** et le **fils droit**.

Définition Récursive

Un arbre binaire est soit vide (NULL), soit formé d'une racine et de deux sous-arbres binaires (gauche et droit).

```
// Structure Nœud Binaire
Structure    NoeudArbre
    valeur : Entier
    gauche : Pointeur sur NoeudArbre
    droite : Pointeur sur NoeudArbre
FinStructure
```

Implémentation d'un nœud d'arbre en mémoire

```
// Création d'un Nœud Binaire
Fonction  creerNoeud (val : Entier ) : Pointeur sur NoeudArbre
  n : Pointeur sur NoeudArbre ← nouveau NoeudArbre ()
  n.valeur ← val
  n.gauche ← NULL
  n.droite ← NULL
  Retourner n
FinFonction
```

Structure à deux branches

Chaque nœud pointe vers deux enfants potentiels.

Si une branche est absente, son pointeur vaut simplement NULL.

Les 3 parcours en profondeur (Depth-First Search - DFS)

1. Préfixe (Pre-Order)

Racine → Gauche → Droite

Traitement du nœud courant avant ses enfants.

2. Infixe (In-Order)

Gauche → Racine → Droite

Donne un tri croissant pour un ABR !

3. Postfixe (Post-Order)

Gauche → Droite → Racine

Traitement des enfants avant leur père (suppression).

Parcours préfixe : Algorithme récursif

```
Procédure  parcoursPrefixe  (r : Pointeur sur NoeudArbre  )  
  Si  r ≠ NULL Alors  
    afficher  (r.valeur) // 1. Traiter Racine  
    parcoursPrefixe  (r.gauche) // 2. Sous-arbre G  
    parcoursPrefixe  (r.droite) // 3. Sous-arbre D  
  FinSi  
FinProcédure
```

Cas d'usage clé

Le parcours préfixe est idéal pour **dupliquer ou cloner un arbre** tout en conservant sa structure exacte.

Parcours infixe : L'ordre trié

```
Procédure  parcoursInfixe  (r : Pointeur sur NoeudArbre  )  
  Si  r ≠ NULL Alors  
    parcoursInfixe  (r.gauche)  
    afficher  (r.valeur)  
    parcoursInfixe  (r.droite)  
  FinSi  
FinProcédure
```

Propriété magique

Appliqué à un Arbre Binaire de Recherche (ABR), le parcours infixe affiche toujours les éléments par **ordre croissant** !

Parcours postfixe : Nettoyage & évaluation

```
Procédure  parcoursPostfixe  (r : Pointeur sur NoeudArbre  )  
  Si  r ≠ NULL Alors  
    parcoursPostfixe  (r.gauche)  
    parcoursPostfixe  (r.droite)  
    afficher  (r.valeur)  
  FinSi  
FinProcédure
```

Cas d'usage clé

- Libérer la mémoire d'un arbre sans créer de pointeurs orphelins.
- Évaluer une expression mathématique sous forme d'arbre.

L'Arbre Binaire de Recherche (ABR / BST)

La Règle d'Or d'un ABR

- Pour tout nœud N :
 - Tous les nœuds du sous-arbre **GAUCHE** ont une valeur $< N$.
 - Tous les nœuds du sous-arbre **DROIT** ont une valeur $> N$.



Exemple d'ABR Valide

Recherche dichotomique dans un ABR

```
Fonction  rechercher (r : Pointeur sur Noeud , val : Entier ) :  
Booleen  
Si r = NULL Alors Retourner Faux FinSi  
  
Si val = r.valeur Alors  
    Retourner Vrai  
SinonSi val < r.valeur Alors  
    Retourner rechercher (r.gauche, val)  
Sinon  
    Retourner rechercher (r.droite, val)  
FinSi  
FinFonction
```

Performance : $O(\log n)$

À chaque comparaison, la moitié de l'arbre restant est éliminée.

Hauteur $h = \log_2(n)$ pour un arbre équilibré.

Insertion dans un ABR en pseudocode

```
Fonction  insererABR (r : Pointeur sur Noeud , val : Entier ) :  
Pointeur sur Noeud  
  Si r = NULL Alors  
    Retourner  creerNoeud (val)  
  FinSi  
  
  Si val < r.valeur Alors  
    r.gauche ← insererABR (r.gauche, val)  
  SinonSi val > r.valeur Alors  
    r.droite ← insererABR (r.droite, val)  
  FinSi  
  
  Retourner  r  
FinFonction
```

Insertion aux Feuilles

L'insertion d'un nouvel élément se fait toujours au niveau d'une feuille disponible en suivant la règle d'ordre.

Suppression dans un ABR : Les 3 Cas

Cas 1

Nœud Feuille

Supprimer simplement le nœud et passer son pointeur père à NULL.

Cas 2

1 Seul Enfant

Relier directement le père de l'élément à l'unique enfant du nœud.

Cas 3

2 Enfants

Remplacer par le minimum du sous-arbre droit (successeur infixe).

Arbre équilibré vs Arbre dégénéré (Peigne)

Arbre Équilibré : $O(\log n)$

Les hauteurs gauche et droite sont équivalentes.

Recherche ultra-rapide.

Arbre Dégénéré (Pire Cas) : $O(n)$

Si l'on insère des éléments déjà triés (1, 2, 3, 4), l'ABR devient une **Liste Chaînée**.

Solution : Arbres AVL ou Red-Black.

Quand utiliser les Arbres Binaires de Recherche ?

Utiliser un ABR Si :

- Vous avez besoin de recherches dynamiques rapides en **$O(\log n)$** .
- Les données changent fréquemment (insertions/suppressions mélangées aux recherches).
- Vous souhaitez maintenir les données constamment triées.

Préférer une Autre Structure Si :

- Vous avez uniquement besoin d'accès direct par index (Tableau).
- Vous effectuez uniquement des recherches exactes par clé sans ordre (Table de Hachage $O(1)$).

Grand tableau comparatif des complexités (Big-O)

Structure	Accès Index	Recherche	Insertion Tête	Insertion Fin
Tableau Statique	$O(1)$	$O(n)$	N/A	N/A
Liste Dynamique	$O(1)$	$O(n)$	$O(n)$	$O(1)$ amorti
Liste Chaînée	$O(n)$	$O(n)$	$O(1)$	$O(1)$ * avec queue
ABR Équilibré	N/A	$O(\log n)$	$O(\log n)$	$O(\log n)$

Exercices d'application